Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming

Laboratory Activity No. 2
Personal MP3 Player
Laboratory Report

Development Environment: PlatformIO, STM32Cube, FreeRTOS, RT-Spark STM32F407ZG
Date: October 13, 2026

Shanice Reih H. Tanque **B182/B187**

1. Problem and Requirements

This laboratory implements a personal music player using the RT-Spark STM32F407ZG board, STM32Cube HAL, FreeRTOS, and PlatformIO. The system provides eight selectable songs, an onboard 240 x 240 ST7789 LCD, ES8388 audio output, potentiometer volume control, external RGB status LEDs, pause/resume control, and a dedicated USER_BUTTON for stop and replay. Song selection uses three pushbuttons as a 3-bit binary value: DOWN is bit 2, LEFT is bit 1, and RIGHT is bit 0. UP captures the current combination. After all selection buttons are released, UP must be pressed again within five seconds to confirm the song. A timeout cancels the pending selection. The final application uses exactly three FreeRTOS application tasks. PollButtonsTask owns control input and player state, AdjustVolumeTask owns ADC sampling and codec volume updates, and UpdateLcdLedsTask owns the runtime LCD and RGB indicators. Audio output remains I2S/DMA driven rather than using a fourth application task. The project does not decode MP3 files. The eight songs are stored as musical note and beat data, converted into synthesized stereo PCM, and streamed through I2S3 and DMA1 Stream5 to the onboard ES8388 codec.

2. System Architecture and Design

2.1 Hardware Architecture

Component	Purpose / Connection
RT-Spark STM32F407ZG	Main controller
ST7789 LCD	Onboard 240 x 240 player interface
ES8388	Onboard audio codec
UP / DOWN / LEFT / RIGHT	PC5 / PC1 / PC0 / PC4; song selection and long-UP control
USER_BUTTON	PA0 / P2 pin 12; active LOW with internal pull-up
Potentiometer	PF6 -> ADC3 Channel 4
External LEDs	Red PA8, Green PA1, Blue PB14
UART1	PA9 / PA10 serial diagnostics

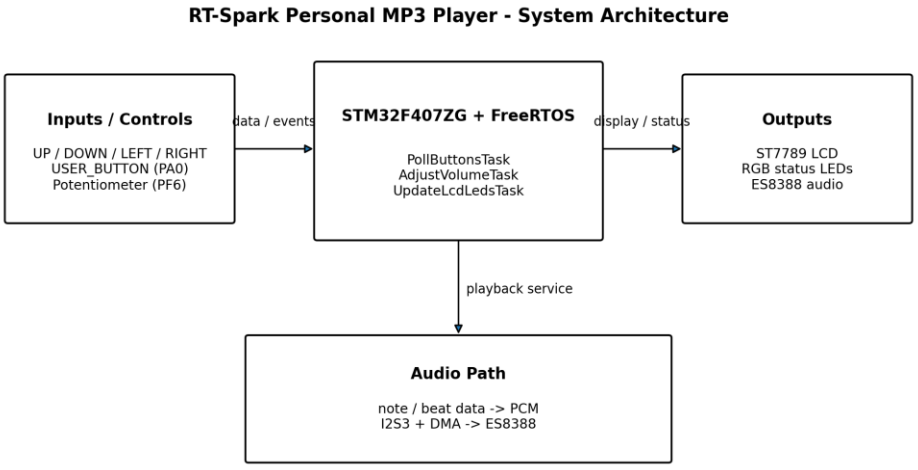


Figure 1. System architecture of the RT-Spark Personal MP3 Player.

2.2 Software Decomposition

The firmware is divided into modules for song selection, audio generation and transport, player-state publication, potentiometer filtering, codec synchronization, LCD synchronization, presentation logic, RGB LED control, and USER_BUTTON debounce/policy. Hardware-independent decision logic is covered by host tests, while the complete firmware is verified on the physical RT-Spark board.

2.3 Playback and Selection State Machine

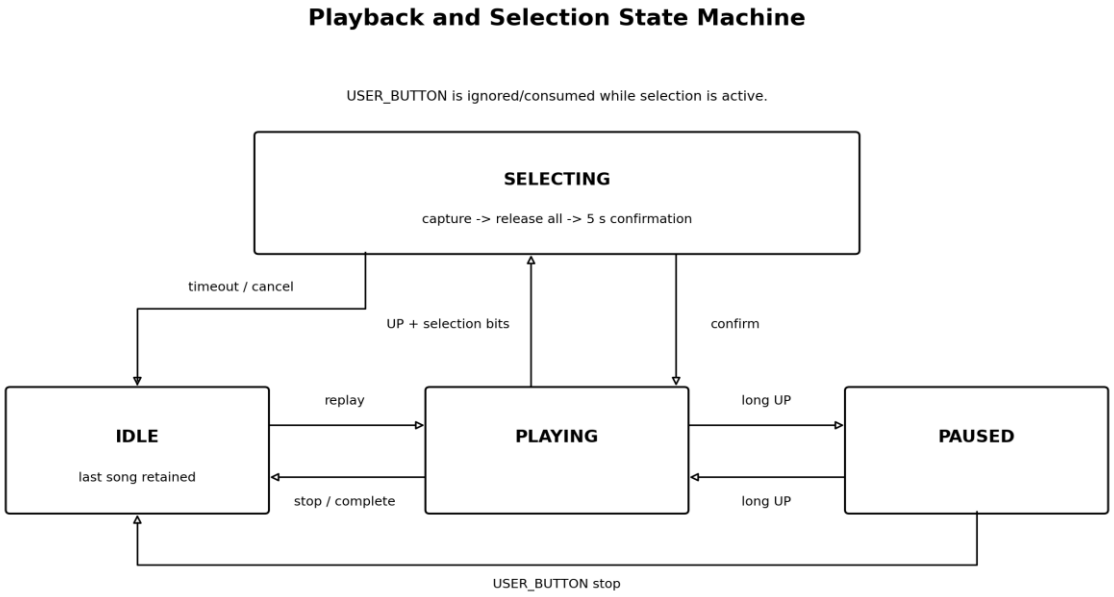


Figure 2. Playback and selection state machine.

Long-UP is reserved for pause/resume and keeps the current playback position. The dedicated USER_BUTTON has separate behavior: it stops a playing or paused song, and a later press from IDLE replays the retained song from the beginning. During active song selection, USER_BUTTON is ignored and consumed so it cannot fire later after confirmation or timeout.

3. FreeRTOS Architecture

Task	Responsibility	Priority	Stack
PollButtonsTask	Selection buttons, USER_BUTTON, player control, audio service, state publication	3	1,024 words
AdjustVolumeTask	PF6 ADC sampling, filter, accepted volume, ES8388 volume writes	2	768 words
UpdateLcdLedsTask	LCD/PlayerUI rendering and external RGB LED state	1	1,024 words

3.1 Priority and Blocking Rationale

PollButtonsTask uses the highest application priority because control input and playback actions require prompt response. AdjustVolumeTask is priority 2 because ADC sampling and codec updates must remain responsive but are less latency-sensitive than control input. UpdateLcdLedsTask uses priority 1 because display rendering can tolerate the most delay. All three tasks block or delay between iterations so lower-priority work still receives CPU time.

3.2 Communication and Synchronization

- Player state is shared through a protected snapshot rather than direct cross-task UI access.
- A state mutex protects short state copies and updates.
- A codec mutex protects ES8388 volume and mute/unmute transactions.
- An LCD mutex protects drawing; UpdateLcdLedsTask remains the only runtime UI owner.
- Audio DMA callbacks remain interrupt-driven and do not call FreeRTOS APIs.
- USER_BUTTON is polled by PollButtonsTask and does not create another task or use EXTI.

4. Implementation

4.1 Song Selection

DOWN, LEFT, and RIGHT form the three-bit song number. A first UP press captures the code, all selection buttons must then be released, and a second UP press confirms within five seconds. The captured candidate remains fixed during the confirmation window.

Binary	Song
000	Fur Elise - Beethoven
001	Canon In D - Pachebelbel
010	Minuet in G major - Bach
011	Turkish March - Mozart
100	Nocturne in E flat - Chopin
101	Waltz No. 2 - Shostakovich
110	Nocturne in C sharp - Chopin
111	Symphony No. 40 - Mozart

4.2 Audio Playback

The song data is converted into 16-bit stereo PCM samples. I2S3 and DMA1 Stream5 continuously supply the ES8388 codec. The audio system supports start, pause, resume, stop, natural completion, and replay from the beginning.

4.3 Potentiometer Volume

PF6 is configured as ADC3 Channel 4 with 12-bit conversion from 0 to 4095. A four-sample filter and deadband reduce jitter. Accepted values are mapped to 0-100% logical volume and written to the ES8388 under codec synchronization. The last valid value is retained if an ADC conversion fails.

4.4 LCD and RGB Status

The final interface uses a purple/lavender theme. The LCD shows startup status, song selection, captured selection, release-all prompt, confirmation countdown, now playing, paused, timeout, and volume overlay screens. The RGB indicators show blue while playing, red while paused, green during selection/confirmation, and all off while idle.

4.5 Dedicated USER_BUTTON

A normally-open pushbutton is connected from PA0 (P2 pin 12) to GND (P2 pin 6). The STM32 internal pull-up makes released HIGH and pressed LOW. The input uses approximately 25 ms debounce, one event per press, release rearming, and boot-held suppression.

A press while PLAYING or PAUSED stops playback and returns the player to IDLE while retaining the last confirmed song. A later press from IDLE replays the same song from the beginning. After natural song completion, that retained song is still replayable. During song selection, USER_BUTTON is suppressed and consumed.

5. Verification and Testing

5.1 Build and Host-Test Validation

Final validation covered all nine PlatformIO environments and fourteen host test suites. All builds and tests passed with zero warnings/errors. The final Stage-6 firmware used 48,384 bytes of Flash and 48,408 bytes of static RAM.

Validation	Result
PlatformIO environments	9/9 passed
Host test suites	14/14 passed
Warnings / errors	0
Final environment	rtos_stage6_user_button
Flash usage	48,384 bytes
Static RAM usage	48,408 bytes

5.2 Physical Functional Verification

ID	Stimulus	Observed Result	Status
FT-01	Select and confirm 000	Fur Elise starts; blue LED / playing screen	PASS
FT-02	Select remaining codes 001-111	Correct songs start for all binary codes	PASS
FT-03	Do not confirm within 5 s	Selection times out and cancels	PASS
FT-04	Long-UP while playing	Pause; long-UP resumes at retained position	PASS
FT-05	Adjust potentiometer	Volume and overlay update	PASS
FT-06	USER_BUTTON while playing	Playback stops; state becomes IDLE	PASS
FT-07	USER_BUTTON while paused	Paused song stops; state becomes IDLE	PASS
FT-08	USER_BUTTON from IDLE	Retained song replays from beginning	PASS
FT-09	USER_BUTTON after natural completion	Last confirmed song replays	PASS
FT-10	Hold USER_BUTTON during selection	Press is consumed; no delayed stop/replay	PASS

5.3 Runtime Health

During Stage-6 physical testing, UART diagnostics showed stable heap usage and positive task stack headroom. A recorded sample showed approximately 598 words free for PollButtonsTask, 722 words for AdjustVolumeTask, 674 words for UpdateLcdLedsTask, and 19,816 bytes for both current and minimum free heap. No HardFault, scheduler freeze, reboot loop, or deadlock was observed.

6. Engineering Discussion

6.1 Key Design Decisions

- The required three-thread structure is preserved: controls, volume, and presentation remain separate application tasks.
- Audio stays DMA/interrupt-driven instead of adding an AudioTask.
- The LCD is single-owner to avoid concurrent display access.
- The potentiometer is the only volume input; LEFT and RIGHT remain selection bits.
- Long-UP pause/resume and USER_BUTTON stop/replay are intentionally separate controls.
- The dedicated button was first verified in an isolated PA0 smoke test before full integration.

6.2 Challenges and Debugging

The main engineering challenge was migrating the working audio player to FreeRTOS without breaking continuous DMA audio. The migration was performed in stages so the scheduler, task ownership, potentiometer control, UI ownership, and USER_BUTTON could each be verified independently. Temporary UART diagnostics were used to trace USER_BUTTON from PA0 through debounce and policy to the audio stop/replay path.

Small ADC changes can appear when the breadboard is physically disturbed. The filter and deadband reduce this effect, but the observed physical potentiometer range remained approximately 8-100% and was intentionally not recalibrated during the final integration.

6.3 Limitations

- Songs are synthesized from note/beat definitions; the firmware does not decode stored MP3 files.
- Only eight songs are exposed through the required 3-bit selector.
- The physical potentiometer range is approximately 8-100% rather than a calibrated full 0-100% electrical range.
- The player has no file-system playlist, persistent storage, or wireless control.
- Heavy UART logging can add timing overhead during development.

7. Conclusion

The RT-Spark Personal MP3 Player demonstrates a complete FreeRTOS-based embedded audio application on the STM32F407ZG. It combines binary song selection, continuous I2S/DMA audio, potentiometer volume, an onboard LCD, external RGB indicators, long-UP pause/resume, and a dedicated PA0 stop/replay button while preserving exactly three application tasks.

Final validation included nine successful firmware builds, fourteen passing host suites, all eight song selections, five-second timeout behavior, pause/resume, potentiometer volume, LCD/RGB presentation, repeated stop/replay, replay after natural completion, and selection-time suppression of USER_BUTTON. Runtime monitoring showed stable heap usage and positive stack headroom during physical testing.

8. Requirements Traceability Matrix

Requirement	Implementation	Verification
8 selectable songs	3-bit mapping 000-111	FT-01 / FT-02
5 s confirmation	Selection controller timeout	FT-03

Pause / resume	Long-UP control	FT-04
Adjustable volume	PF6 / ADC3 Channel 4	FT-05
LCD feedback	UpdateLcdLedsTask / PlayerUI	Physical tests
RGB state indication	PA8 / PA1 / PB14	Physical tests
Dedicated stop / replay	PA0 USER_BUTTON	FT-06 to FT-10
Three RTOS threads	PollButtons / AdjustVolume / UpdateLcdLeds	Build + runtime health

9. Hardware and Demonstration Evidence

Replace the following placeholders with your own final photographs/screenshots before submission.

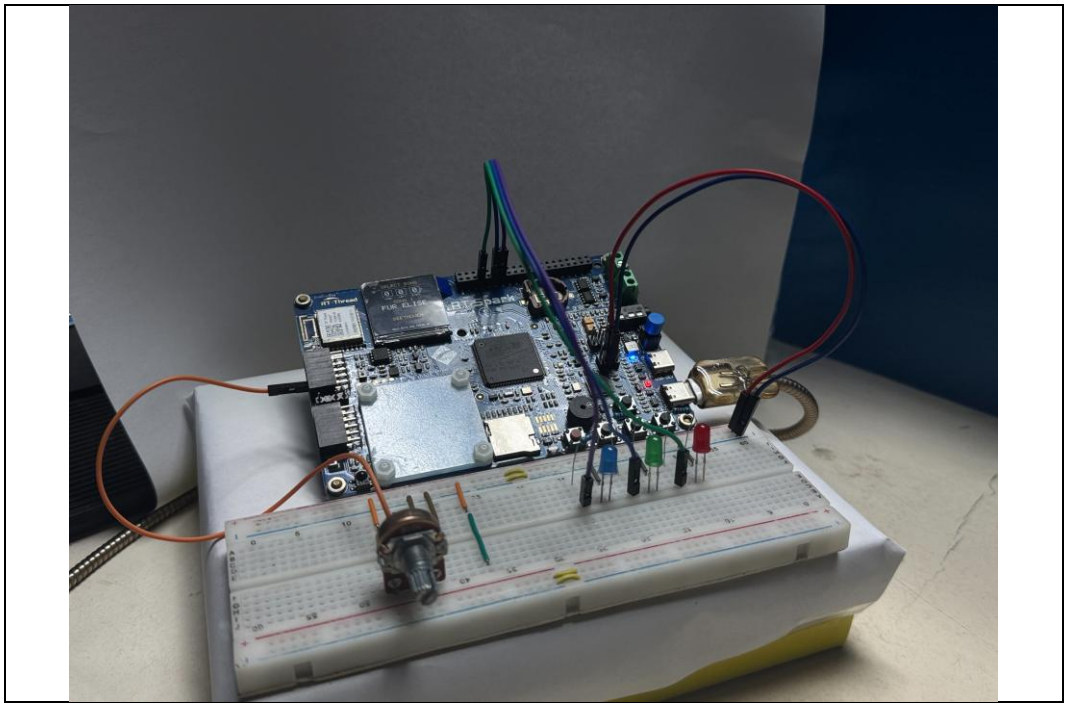


Figure 3. Final RT-Spark Personal MP3 Player hardware setup.

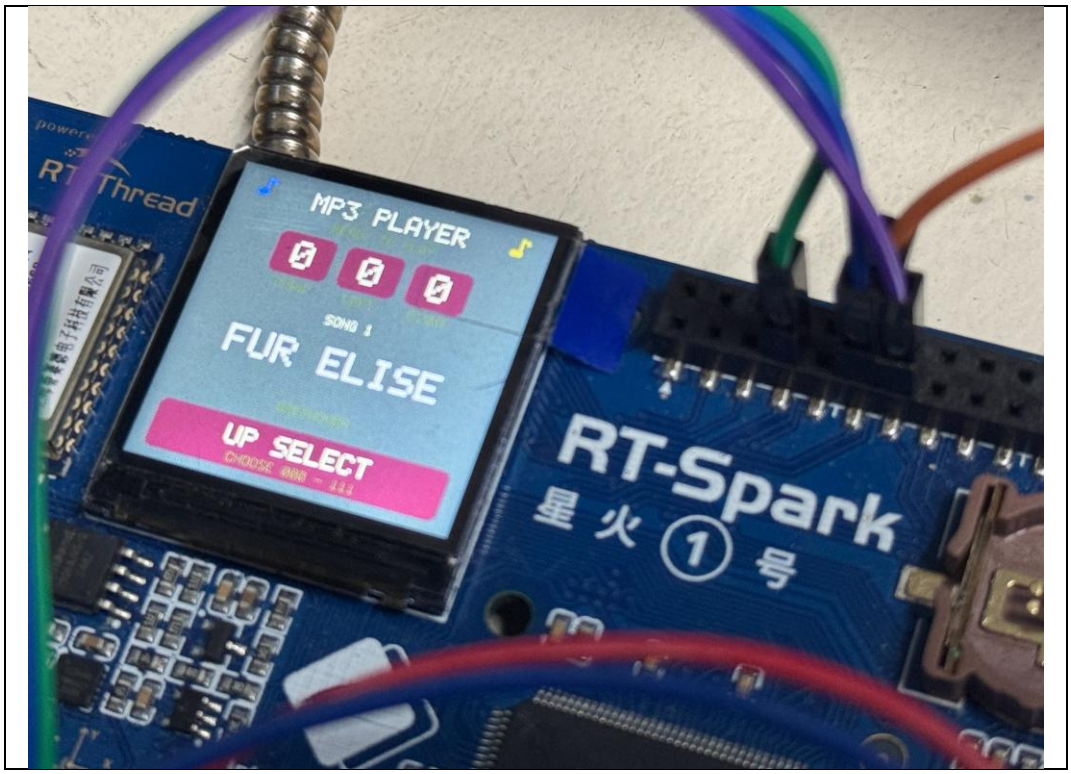


Figure 4. Purple PlayerUI during physical playback verification.

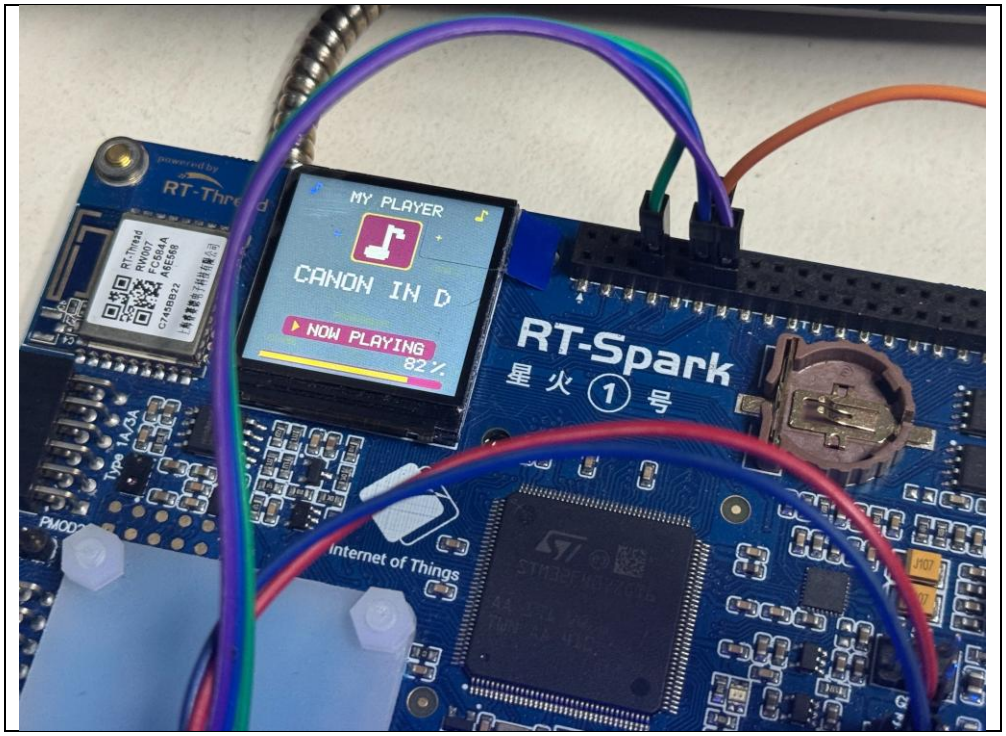


Figure 5. UART evidence of USER_BUTTON stop/replay and RTOS health.

References

- BCA182 Embedded Systems Programming - Laboratory Activity 2: Personal MP3 Player.
- FreeRTOS documentation.
- STM32CubeF4 HAL documentation.
- PlatformIO documentation.
- RT-Spark STM32F407ZG board resources.
- ES8388 audio codec documentation.